



Cairo University

Faculty of Engineering

Department of Computer Engineering

Accelera AutoML



Automated Machine-Learning Pipeline Selection
and Optimization in Accelera

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Mohamed Khater

ID: 9220713

Supervised by

Dr. Salma Abdelmonem

June 2026

All rights reserved. This report may not be reproduced in whole or in part, by photocopying
or other means, without the permission of the authors/department.

Abstract

Selecting an appropriate machine-learning pipeline, including data preprocessing, algorithm selection, and hyperparameter optimization, is a complex and computationally expensive process. This thesis presents Accelera AutoML, an automated machine-learning (AutoML) module integrated into the Accelera framework. The proposed module provides a unified scikit-learn-style interface for both classification and regression tasks while automating the construction and optimization of machine-learning pipelines.

To efficiently explore the search space, Accelera AutoML combines conditional configuration spaces, cross-validated model evaluation, multi-fidelity optimization, and Bayesian optimization using a random-forest surrogate model with an expected-improvement acquisition function. The system further incorporates meta-learning warm starts, timeout-based isolation of candidate evaluations, and ensemble construction through voting and stacking techniques.

This thesis describes the design, architecture, implementation, and evaluation of Accelera AutoML, demonstrating how these components can be integrated into a practical AutoML system that reduces manual effort while producing competitive machine-learning models.

الملخص

يُعد اختيار خط أنابيب مناسب للتعلم الآلي، بما في ذلك معالجة البيانات المسبقة، واختيار الخوارزمية، وضبط المعاملات الفائقة، عملية معقدة ومكلفة حسابياً. تقدم هذه الرسالة نظام Accelera AutoML، وهو وحدة للتعليم الآلي المؤتمت مدججة داخل إطار عمل Accelera. توفر الوحدة المقترحة واجهة موحدة تشبه واجهة scikit-learn لكل من مهام التصنيف والانحدار، مع أتمتة بناء خطوط أنابيب التعلم الآلي وتحسينها.

لاستكشاف فضاء البحث بكفاءة، يجمع نظام Accelera AutoML بين فضاءات التهيئة الشرطية، وتقييم النماذج باستخدام التحقق المتقاطع، والتحسين متعدد الدقة، والتحسين البليزي باستخدام نموذج بديل قائم على الغابة العشوائية مع دالة اكتساب للتحسن المتوقع. كما يدمج النظام بدايات تمهيدية قائمة على التعلم الفوقي، وعزل تقييمات النماذج المرشحة باستخدام حدود زمنية، وبناء التجميعات باستخدام تقنيات التصويت والتكديس.

تصف هذه الرسالة تصميم نظام Accelera AutoML وبنيته وتنفيذه وتقييمه، وتوضح كيف يمكن دمج هذه المكونات في نظام عملي للتعليم الآلي المؤتمت يقلل الجهد اليدوي مع إنتاج نماذج تعلم آلي تنافسية.

Acknowledgment

The author would like to express sincere gratitude to Dr. Salma Abdelmonem for her supervision, guidance, and support throughout this project. The author also thanks the Computer Engineering Department and everyone who supported the design, implementation, and evaluation of this work.

Contents

Abstract	i
المخلص	ii
Acknowledgment	iii
List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
List of Symbols	x
Contacts	xi
1 Introduction	1
1.1 Motivation and Justification	1
1.2 Project Objectives and Problem Definition	1
1.3 Project Need	2
1.4 Project Objectives	2
1.5 Project Outcomes	3
1.6 Scope	3
1.7 Document Organization	4
2 Market Visibility Study	5
2.1 Target Customers	5
2.2 Market Survey	5
2.3 Business Case and Financial Analysis	5

3	Literature Survey	6
3.1	Combined Algorithm Selection and Hyperparameter Optimization	6
3.2	Search Strategies	6
3.3	Multi-Fidelity Evaluation	7
3.4	Meta-Learning for Warm Starts	7
3.5	Ensemble Learning	8
3.6	Related AutoML Systems	8
4	System Design and Architecture	9
4.1	Overview and Assumptions	9
4.2	System Architecture	10
4.2.1	Block Diagram	10
4.3	Functional Requirements	12
4.4	Non-Functional Requirements	12
4.5	Constraints and Assumptions	13
4.6	Accelera AutoML Module Design	13
4.6.1	Method Overview	13
4.6.2	Configuration-Space Construction	14
4.6.3	Evaluation and Cost Conversion	15
4.6.4	Three-Stage Fidelity Schedule	17
4.6.5	Surrogate-Guided Candidate Selection	18
4.6.6	Warm-Start Selection	21
4.6.7	Parallel Execution and Timeout Control	22
4.6.8	Final Model and Ensemble Selection	23
4.7	Chapter Summary	25
5	System Testing and Verification	26
5.1	Testing Setup	26
5.2	Testing Plan and Strategy	26
5.3	Comparative Results to Previous Work: Classification	27

5.4	Comparative Results to Previous Work: Regression	30
5.5	Result Reproduction	32
5.6	Chapter Summary	32
6	Conclusions and Future Work	33
6.1	Faced Challenges	33
6.2	Gained Experience	33
6.3	Conclusions	33
6.4	Future Work	34
A	Development Platforms and Tools	35
A.1	Development Platform	35
A.2	Software Tools	35
A.2.1	ConfigSpace	35
B	Use Cases	36
C	User Guide	37
D	Code Documentation	38
E	Feasibility Study	40

List of Figures

4.1	Block diagram of the Accelera AutoML module.	11
4.2	Simplified promotion flow in the three-stage fidelity schedule	18
4.3	Surrogate-guided candidate-selection workflow	21
4.4	Warm-start transfer from similar historical datasets	22
4.5	Process-level parallel execution and timeout handling	23
5.1	Mean classification accuracy for the evaluated systems.	27
5.2	Mean classification runtime for the evaluated systems.	28
5.3	Per-dataset classification accuracy on common datasets.	28
5.4	Per-dataset classification duration on common datasets.	29
5.5	Mean regression R^2 for the evaluated systems.	30
5.6	Mean regression runtime for the evaluated systems.	31
5.7	Per-dataset regression R^2 on common datasets.	31
5.8	Per-dataset regression duration on common datasets.	32

List of Tables

3.1	Selected AutoML systems and their primary design emphasis	8
4.1	Functional requirements	12
4.2	Example of conditional hyperparameter activation	15
4.3	Examples of metric-to-cost conversion	16
4.4	Default evaluation fidelity stages	17
4.5	Main quantities used in surrogate-guided candidate selection	20
5.1	Classification benchmark summary.	27
5.2	Updated regression benchmark summary over ten datasets.	30
D.1	Main implementation units of Accelera AutoML	38

List of Abbreviations

AI	Artificial Intelligence
AMLB	AutoMLBenchmark
API	Application Programming Interface
AutoML	Automated Machine Learning
CV	Cross-Validation
EI	Expected Improvement
ML	Machine Learning
RMSE	Root Mean Squared Error
SMBO	Sequential Model-Based Optimization

List of Symbols

λ	Hyperparameter configuration
$\mathcal{A}$	Set of candidate algorithms
D	Dataset
f	Fidelity tuple
s	Fidelity stage index
q	Sample fraction
k	Cross-validation fold count
b	Model budget multiplier
μ	Surrogate predicted mean cost
σ	Surrogate predicted cost standard deviation

Contacts

Team Members

Name	Email	Phone Number
Mohamed Abdelaziz Khater	mk0015264@gmail.com	+201068071299

Supervisor

Name	Email	Number
Dr. Salma Abdelmonem	salma.cufece@gmail.com	+201114302362

This page is left intentionally empty.

Chapter 1: Introduction

Accelera AutoML is an automated machine-learning module developed for the Accelera framework. The project addresses the practical difficulty of selecting and tuning machine-learning pipelines for tabular classification and regression tasks. It provides a unified estimator interface while automating model selection, hyperparameter optimization, fidelity scheduling, warm-start selection, and ensemble construction.

1.1 Motivation and Justification

Machine-learning development is rarely limited to fitting one estimator. A practical workflow requires the user to select a model family, define its hyperparameters, decide whether features require scaling, choose an evaluation metric, allocate computational resources, and compare alternatives without leaking validation information. These decisions interact. For example, a support-vector machine may benefit from feature scaling, while a tree-based model usually does not require it. A configuration that performs well on a small sample may fail when trained at full fidelity, and a highly accurate model may be unsuitable if its search cost exceeds the available budget.

Automated machine learning (AutoML) treats these choices as a structured optimization problem. Instead of manually testing a short list of models, an AutoML system defines a search space and evaluates configurations under a consistent protocol. The system must balance exploration of new regions with exploitation of observations collected during the current search. It must also handle practical failures such as invalid parameter combinations, estimator exceptions, and trials that exceed their time limit.

1.2 Project Objectives and Problem Definition

The central problem addressed by this work is:

How can Accelera automatically produce a strong classification or regression model under explicit trial and time constraints, while reducing unnecessary full-cost evaluations and exposing a familiar estimator interface?

This problem contains several subproblems:

1. representing diverse hyperparameter spaces;
2. comparing configurations using a fair task-specific evaluation protocol;
3. allocating computational resources across low and high-fidelity evaluations efficiently;
4. transferring useful configurations from related historical datasets;
5. selecting an effective final model or ensemble;
6. isolating estimator failures so that individual candidates cannot terminate the entire search process.

1.3 Project Need

Manual model selection is difficult to reproduce because it often depends on informal trial-and-error decisions. It is also inefficient when many model families have large hyperparameter spaces. A reusable AutoML component provides three practical benefits. First, all candidates are evaluated by the same metric and validation policy. Second, resource limits become explicit through parameters such as `time_budget`, `n_trials`, and `per_run_time_limit`. Third, the complete run history can be exposed as a leaderboard for inspection rather than returning an unexplained model.

The module is implemented inside Accelera instead of as an isolated script so that it can share the framework's packaging, preprocessing, reporting, and future pipeline capabilities. The public API follows the fit/predict conventions of scikit-learn, reducing the learning cost for users familiar with common Python machine-learning libraries.

1.4 Project Objectives

The aim of this work is to design and implement a resource-aware AutoML engine for tabular classification and regression. Its objectives are:

- provide `AutoMLClassifier` and `AutoMLRegressor`;
- search multiple model families and their conditional parameters;
- support user-defined scoring, cross-validation, and reproducible seeds;

- evaluate candidates at progressively higher fidelity;
- use observed trials to guide later sampling;
- warm-start search using dataset metafeatures and stored historical runs;
- construct voting or stacked ensembles from competitive candidates;
- support parallel trials and per-trial timeout isolation;
- return the final model, best score, run history, and leaderboard.

1.5 Project Outcomes

The current implementation makes the following engineering and methodological contributions:

1. A shared engine for classification and regression with task-specific evaluators and component registries.
2. A three-stage fidelity schedule that varies sample fraction, cross-validation folds, and estimator budget.
3. A sequential model-based optimizer using a random-forest surrogate and expected improvement over a sampled candidate pool.
4. A promotion mechanism that advances successful low-fidelity candidates according to their relative cost.
5. Classification and regression metafeatures for retrieving warm-start configurations from packaged metadata.
6. Voting and stacked ensemble construction, including forward selection of diverse base models.

1.6 Scope

This thesis focuses on supervised tabular classification and regression. The implemented system assumes data that can be validated by scikit-learn's array validation utilities. The work does not currently claim support for automated deep neural architecture search, unsupervised learning, time-series-specific validation, streaming updates, or distributed cluster execution. These are possible extensions but are outside the present implementation scope.

1.7 Document Organization

Chapter 2 presents the market visibility study. Chapter 3 introduces the technical foundations and related AutoML systems. Chapter 4 describes requirements, design decisions, implementation architecture, and the implemented approach. Chapter 5 presents testing and benchmark verification. Chapter 6 concludes the report and outlines future work.

Chapter 2: Market Visibility Study

This chapter discusses the market context for Accelera AutoML and explains why an Accelera-native AutoML component is useful. It identifies the expected users of the module, surveys related AutoML tools, and summarizes the practical value of reducing repeated manual model-selection effort.

2.1 Target Customers

The target customers for Accelera AutoML are data scientists, machine-learning engineers, researchers, and software developers who need a competitive tabular classification or regression baseline without manually testing many estimator families and hyperparameter settings.

2.2 Market Survey

Several AutoML tools address similar needs. Auto-sklearn combines Bayesian optimization, meta-learning, and ensembles. FLAML emphasizes low-cost search under limited resources. TPOT uses evolutionary pipeline search, while AutoGluon Tabular focuses on strong stacked ensembles through a high-level interface. Accelera AutoML is positioned as an Accelera-native alternative that keeps the public estimator interface familiar while exposing internal search, fidelity, and ensemble decisions for project-specific extension.

2.3 Business Case and Financial Analysis

The business value of Accelera AutoML comes from reducing the engineering time required to establish reliable machine-learning baselines. Instead of assigning a developer to manually compare preprocessing choices, models, and hyperparameters, a project can run a bounded AutoML search and inspect the resulting leaderboard. Operational costs are limited because the module runs locally inside the user environment and does not require a hosted service. Potential value increases when the same module is reused across many Accelera projects, where each saved modeling cycle reduces repeated manual effort.

Chapter 3: Literature Survey

This chapter introduces the technical background needed to understand the project. It covers combined algorithm selection and hyperparameter optimization, search strategies, multi-fidelity evaluation, meta-learning, ensemble learning, and related AutoML systems. The chapter ends by positioning Accelera AutoML relative to existing tools and explaining the concepts used in the implemented approach.

3.1 Combined Algorithm Selection and Hyperparameter Optimization

Given a set of algorithms $\mathcal{A}$ and a hyperparameter domain Λ_a for every algorithm a , combined algorithm selection and hyperparameter optimization seeks

$$(a^*, \lambda^*) = \arg \min_{a \in \mathcal{A}, \lambda \in \Lambda_a} \widehat{L}(a, \lambda; D), \quad (3.1)$$

where $\widehat{L}$ is an estimated validation loss on dataset D . Auto-WEKA formalized this combined view by searching over both learning algorithms and their hyperparameters [1]. The domain is conditional: parameters belonging to a random forest are inactive when the selected model is logistic regression. This conditional structure motivates the use of the ConfigSpace representation in Accelera AutoML.

3.2 Search Strategies

Grid search evaluates a fixed Cartesian product and scales poorly as the number of dimensions increases. Random search allocates trials more effectively when only a subset of hyperparameters strongly affects performance [7]. Sequential model-based optimization improves on uninformed sampling by fitting a surrogate to observed configuration costs. SMAC uses a random-forest model for algorithm configuration problems and selects candidates through an acquisition function [6].

Accelera AutoML follows this general model-based pattern. It first evaluates initial and warm-start configurations, represents each configuration as a numeric vector, augments that vector with fidelity variables, and fits a random-forest regressor.

Predictions from individual trees provide an empirical mean and standard deviation. Expected improvement then ranks a randomly sampled candidate pool. The implementation is inspired by SMAC principles but is a project-specific optimizer rather than a dependency on the external SMAC package.

3.3 Multi-Fidelity Evaluation

Full cross-validation on an entire dataset is expensive, particularly for large ensembles or boosting models. Multi-fidelity optimization first evaluates candidates using cheaper approximations. Fidelity may be varied by the number of training instances, the number of validation folds, training iterations, or model capacity. Poor candidates can be discarded early, while promising ones receive more resources.

In this work, fidelity is represented by a tuple

$$f = (s, q, k, b), \tag{3.2}$$

where s is the stage index, q is the sample fraction, k is the number of cross-validation folds, and b is a multiplicative model budget. The fidelity vector is included in the surrogate input so that observations from different stages remain distinguishable.

3.4 Meta-Learning for Warm Starts

Meta-learning attempts to transfer knowledge across machine-learning tasks. An AutoML system can characterize a new dataset using metafeatures, compare it with previously evaluated datasets, and schedule successful historical configurations early. Auto-sklearn combines Bayesian optimization, meta-learning, and ensemble construction [2]. The intended benefit is not to guarantee that a historical configuration remains optimal, but to reduce the cold-start period before useful observations are available.

Accelera AutoML computes task-dependent statistical descriptors. Shared descriptors include the number of instances and features, numeric/categorical ratios, missing-value ratios, skewness, kurtosis, and categorical symbol statistics. Classification additionally records class entropy, class-count statistics, and class probabilities. Regression records target mean, standard deviation, skewness, and kurtosis. Stored datasets are ranked by distance in this metafeature space, and candidate configurations from the closest datasets are converted back into active ConfigSpace objects.

3.5 Ensemble Learning

Ensembles combine several fitted models to reduce the risk of selecting one unstable estimator. Voting uses the predictions or probabilities of multiple base estimators. Stacked generalization instead trains a meta-model on out-of-fold predictions from base models [8]. Out-of-fold generation is essential: fitting the meta-model on predictions made on each base model's own training observations would produce leakage and an overly optimistic result.

The Accelera implementation provides both strategies. Stacking uses bagged base estimators, creates out-of-fold prediction blocks, performs forward selection, and fits a logistic-regression meta-model for classification or a ridge-regression meta-model for regression. The original feature matrix may optionally be appended to the prediction blocks.

3.6 Related AutoML Systems

Table 3.1: Selected AutoML systems and their primary design emphasis

System	Design emphasis
Auto-WEKA	Combined algorithm selection and hyperparameter optimization over WEKA components [1].
auto-sklearn	Bayesian optimization, dataset metafeatures, and post-hoc ensembles for scikit-learn workflows [2].
TPOT	Evolutionary search over tree-structured machine-learning pipelines [3].
FLAML	Cost-aware, lightweight AutoML designed to find strong configurations under limited resources [4].
AutoGluon Tabular	Multi-layer stacking and ensembling with a high-level tabular prediction interface [5].
Accelera AutoML	Conditional model spaces, staged fidelity, surrogate-guided sampling, packaged warm starts, and selectable voting or stacking within Accelera.

The objective of Accelera AutoML is not to reproduce every capability of these frameworks. Its role is to provide an understandable, extensible AutoML engine that integrates directly with the Accelera codebase and exposes its search state for later research and experimentation.

Chapter 4: System Design and Architecture

The Accelera AutoML module was designed as a resource-aware AutoML workflow for supervised tabular classification and regression. The design starts from a simple user-facing estimator interface, then builds an internal search process that validates the input data, constructs a conditional configuration space, evaluates candidate pipelines, and selects a final fitted model. The implementation was built around separate internal responsibilities for configuration-space construction, candidate evaluation, optimization, meta-learning warm starts, timeout handling, and ensemble selection so that each part can be tested and extended without changing the public interface.

4.1 Overview and Assumptions

Accelera AutoML automates supervised tabular machine-learning pipeline selection for classification and regression tasks. The module receives a dataset and user configuration, searches over valid model and hyperparameter configurations, evaluates candidates under resource limits, and returns a fitted final model together with inspection outputs such as the leaderboard and run history.

The primary user is a data scientist or developer who has a supervised dataset and wants a competitive baseline without manually enumerating estimators. A secondary user is a researcher who needs access to the run history, leaderboard, fidelity stages, and ensemble decisions. A maintainer must be able to add a model family without rewriting the optimization engine.

The main use cases are:

- fit a classifier and obtain labels or probabilities;
- fit a regressor and obtain numeric predictions;
- restrict search to an allowed subset of model families;
- select a metric and cross-validation fold count;
- limit the complete search or each individual trial;
- enable or disable meta-learning and ensembles;

- inspect the best score, leaderboard, and complete run history;

The design assumes that the input data can be represented as tabular feature and target arrays.

4.2 System Architecture

At a high level, Accelera AutoML follows a pipeline architecture. The user interacts with a public estimator, the estimator validates the input data, the AutoML engine builds a task-specific search space, and the optimizer repeatedly selects candidate configurations for evaluation. The evaluator scores candidates using cross-validation and returns structured results to the optimizer. Meta-learning can provide warm-start configurations, and the ensemble stage can combine strong candidates before final model selection.

4.2.1 Block Diagram

Figure [4.1](#) shows the main blocks of the Accelera AutoML module and the flow between user input, search, evaluation, and final output.

Simplified AutoML Workflow

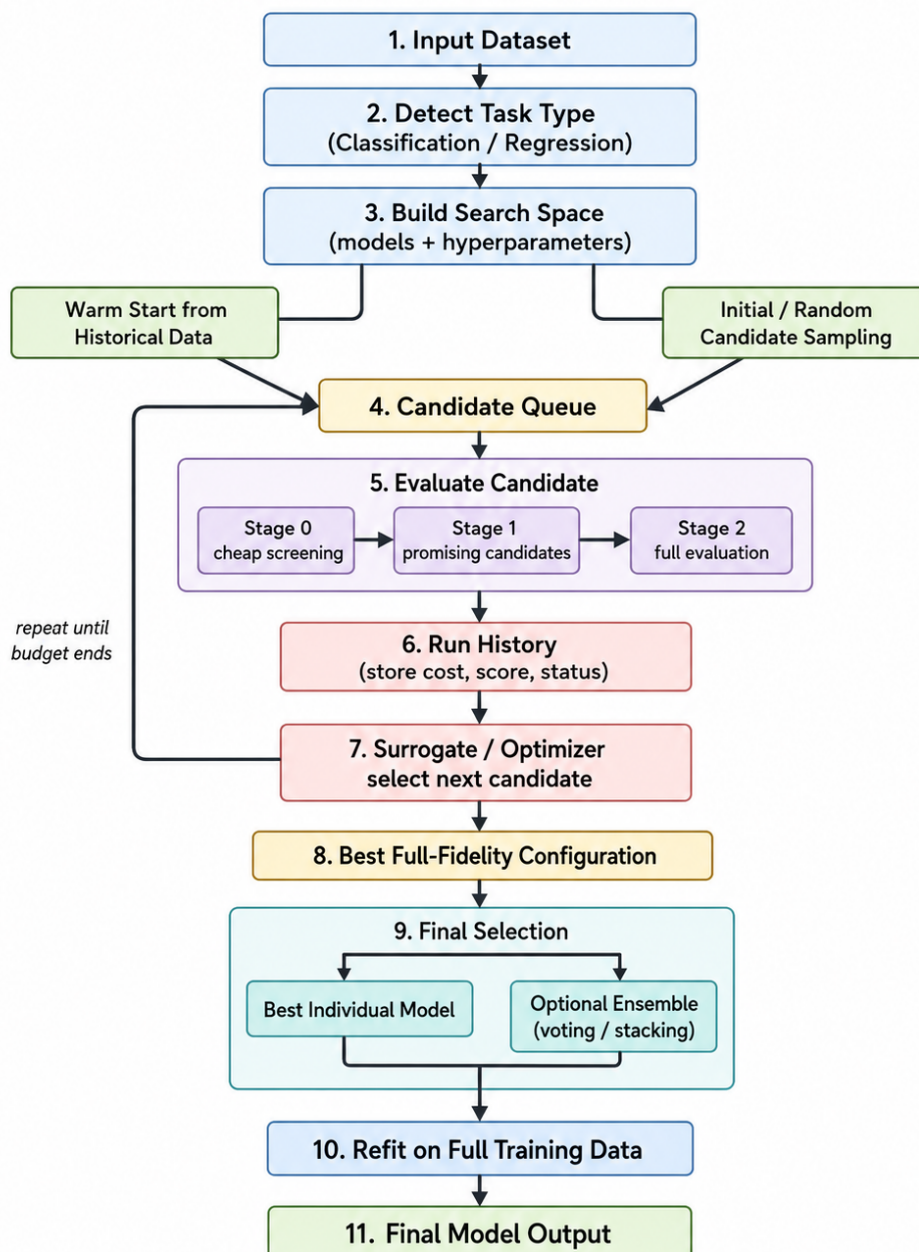


Figure 4.1: Block diagram of the Accelera AutoML module.

4.3 Functional Requirements

Table 4.1: Functional requirements

ID	Requirement
FR-01	The system shall support supervised classification and regression.
FR-02	The public estimators shall provide <code>fit</code> , <code>predict</code> , and <code>score</code> ; classification shall also provide <code>predict_proba</code> when supported by the selected final model.
FR-03	The system shall validate input arrays before fitting or prediction.
FR-04	The engine shall construct a task-specific conditional configuration space from registered components.
FR-05	The evaluator shall compare configurations using cross-validation and the selected scoring function.
FR-06	The optimizer shall stop when the trial limit or overall time budget is reached.
FR-07	Trials shall support staged fidelity and promotion to higher stages.
FR-08	The search shall optionally begin with meta-learning warm starts.
FR-09	The engine shall optionally construct voting or stacked ensembles.
FR-10	The final result shall include a fitted final model, score, leaderboard, ensemble reference, and run history.
FR-11	Trial exceptions and timeouts shall be recorded instead of terminating the entire optimization run.
FR-12	Independent trials shall support process-level parallel execution.

4.4 Non-Functional Requirements

- Usability: API behavior should resemble scikit-learn estimators and use explicit keyword parameters.
- Reproducibility: stochastic estimators, data splitting, and search sampling should receive the configured random state where supported.
- Extensibility: component definitions, evaluator behavior, optimization, meta-learning, and ensembles should remain separate.
- Fault isolation: long-running trials should be terminable without corrupting the parent search process.

- Inspectability: failed and successful trials should remain in the run history with status, duration, score, cost, and fidelity data.

4.5 Constraints and Assumptions

The current implementation is constrained by the interfaces of the underlying estimators.

The evaluator assumes independent and identically distributed tabular samples. Classification uses stratified splitting where possible, while regression uses ordinary K-fold or shuffle splitting. Specialized grouped, temporal, or spatial validation is outside the current design.

4.6 Accelera AutoML Module Design

4.6.1 Method Overview

The complete method is organized as the following sequence:

1. validate the feature matrix and target;
2. resolve compatible or user-allowed model families;
3. construct the conditional configuration space;
4. compute dataset metafeatures and retrieve warm-start configurations;
5. evaluate initial configurations at low fidelity;
6. fit a surrogate after sufficient finite observations exist;
7. select new candidates by expected improvement;
8. promote strong candidates through higher fidelity stages;
9. fit the best full-fidelity individual configuration;
10. optionally construct and evaluate an ensemble.
11. return whichever fitted candidate has the stronger validation score.

4.6.2 Configuration-Space Construction

The configuration space defines the set of machine-learning pipelines that the AutoML engine is allowed to explore. In this work, classification and regression use independent component registries because the available estimators and their valid hyperparameters differ between task types. Each component in a registry specifies four main elements: a model name, a hyperparameter factory, conditional rules, and an estimator builder.

The global configuration contains a categorical variable called `model_name`. This variable represents the learning algorithm selected for a trial. After a model is selected, only the hyperparameters belonging to that model are activated. Hyperparameters belonging to other models remain inactive and are ignored during evaluation. This conditional structure prevents the optimizer from treating irrelevant parameters as active decisions. For example, the number of neighbors is meaningful for a k-nearest-neighbors model, but it is not meaningful for a random forest or a support-vector machine.

Formally, let the set of candidate models be

$$\mathcal{M} = \{m_1, m_2, \dots, m_k\}.$$

Each model m_i has its own hyperparameter space Λ_i . The full conditional configuration space can be written as

$$\Lambda = \bigcup_{i=1}^k (\{\text{model_name} = m_i\} \times \Lambda_i).$$

This formulation means that each valid configuration consists of two parts: the selected model and the hyperparameters that are valid for that selected model. Therefore, the optimizer searches only within the subspace associated with the chosen estimator, instead of searching over all possible hyperparameters at the same time.

Table 4.2: Example of conditional hyperparameter activation

Selected model	Active hyperparameters	Inactive hyperparameters
Random forest	number of trees, maximum depth, split criterion	number of neighbors, kernel type
K-nearest neighbors	number of neighbors, distance metric, weighting strategy	number of trees, maximum depth
Support-vector machine	kernel type, regularization parameter, gamma	number of neighbors, tree depth

Before constructing the final search space, the engine resolves the allowed model set. The user may provide this set explicitly when only specific algorithms should be considered. If no explicit set is provided, the engine starts from the default registry for the task. The engine can then remove incompatible models based on dataset properties. For example, some algorithms assume non-negative feature values and can be excluded when the training data contains negative values.

Restricting the search space has two main benefits. First, it reduces invalid trials because the optimizer is less likely to sample configurations that cannot be evaluated successfully. Second, it allows domain knowledge to be included without changing the optimizer itself. The optimizer only receives the final valid configuration space, while model compatibility and filtering rules are handled by the AutoML engine. This separation makes the system easier to extend, because new models can be added by registering their hyperparameters, conditions, and estimator builder without rewriting the search procedure.

4.6.3 Evaluation and Cost Conversion

After a configuration is sampled, the evaluator is responsible for estimating its predictive performance. For a configuration λ , the engine first builds the corresponding estimator and determines the preprocessing strategies that are allowed for the selected model. The evaluator then tests each model-preprocessor pair using cross-validation. Among all successful preprocessing choices, the pair with the best validation performance is retained as the evaluation result for λ .

Let $m(\lambda)$ denote the mean validation metric obtained by cross-validation. Since different metrics have different optimization directions, the evaluator converts the metric value into a cost value $c(\lambda)$. The optimizer always minimizes this cost, so smaller values indicate better configurations.

For metrics that are naturally minimized, such as mean squared error or mean absolute error, the cost can be used directly:

$$c(\lambda) = m(\lambda).$$

For metrics that are naturally maximized, such as accuracy, F_1 -score, or R^2 , the evaluator converts the score into a minimization objective. A common conversion is

$$c(\lambda) = 1 - m(\lambda),$$

when the metric is bounded between 0 and 1. For metrics that are not strictly bounded in this range, the evaluator may instead use a negated form:

$$c(\lambda) = -m(\lambda).$$

This conversion gives the optimizer a consistent objective regardless of the original metric. In all cases, minimizing $c(\lambda)$ corresponds to improving the validation performance of the configuration.

Table 4.3: Examples of metric-to-cost conversion

Metric	Original direction	Example cost conversion
Accuracy	Maximize	$c(\lambda) = 1 - m(\lambda)$
F_1 -score	Maximize	$c(\lambda) = 1 - m(\lambda)$
R^2 score	Maximize	$c(\lambda) = -m(\lambda)$
Mean squared error	Minimize	$c(\lambda) = m(\lambda)$
Mean absolute error	Minimize	$c(\lambda) = m(\lambda)$

If an evaluation fails, the evaluator does not allow the error to terminate the complete AutoML search. Instead, the failed configuration receives a predefined failure cost and the error message is stored in the run history. This allows the optimizer to continue exploring other configurations while still recording which trials were invalid or unsuccessful.

The validation procedure depends on the learning task. For classification, stratified cross-validation is used so that each fold preserves the class distribution as much as possible. Stratified subsampling is also used when only a fraction of the training data is evaluated at a lower fidelity. This is important because random subsampling may accidentally remove minority classes from small subsets.

For regression, ordinary K-fold cross-validation is used because there are no discrete class labels to preserve. Random subsampling is also sufficient for regression tasks. In classification, the requested number of folds may be adjusted when the smallest class does not contain enough samples to support the requested fold count. For example, if the smallest class contains only three samples, then using five stratified folds is not valid. In this case, the classifier reduces the fold count to a feasible value. This prevents invalid cross-validation splits and improves the robustness of the evaluation process.

4.6.4 Three-Stage Fidelity Schedule

To reduce the cost of the AutoML search, the engine evaluates configurations using a three-stage fidelity schedule. Instead of evaluating every configuration using the full dataset, the full number of cross-validation folds, and the full model capacity, the system begins with a cheaper approximation. Only configurations that perform well at cheaper stages are promoted to more expensive stages.

The default fidelity schedule is shown in Table 4.4.

Table 4.4: Default evaluation fidelity stages

Stage	Data	Folds	Budget	Purpose
0	20%	2	25%	Inexpensive screening of many configurations.
1	50%	3	60%	Intermediate confirmation of promising configurations.
2	100%	Configured value	100%	Full-fidelity ranking and final comparison.

Let $s \in \{0, 1, 2\}$ denote the fidelity stage. Each stage is described by three values: the sample fraction ρ_s , the number of cross-validation folds k_s , and the model budget b_s . Therefore, the trial specification at stage s can be written as

$$\tau_s = (\rho_s, k_s, b_s).$$

The sample fraction controls how much of the training data is used during evaluation. The number of folds controls the reliability of the validation estimate. The model budget controls supported model-capacity parameters, such as the number of estimators in ensemble models or the number of training iterations in iterative models. Early stages are therefore cheaper because they use less data, fewer folds, and smaller model

budgets.

The evaluator scales supported capacity parameters using the model budget. If θ is a capacity parameter and $\theta_{\max}$ is the value requested by the sampled configuration, the stage-specific value can be written as

$$\theta_s = \max(\theta_{\min}, \lfloor b_s \theta_{\max} \rfloor),$$

where $\theta_{\min}$ is the minimum valid value for that parameter. This keeps the reduced model valid while still preserving the relative capacity requested by the configuration.

A newly sampled configuration is first evaluated at stage zero. After a successful evaluation, the optimizer stores the observed cost in a per-configuration state. Once enough observations are available at a given stage, the optimizer ranks configurations according to their costs. Since the optimizer minimizes cost, smaller values are better. Only configurations inside the configured top promotion quantile are advanced to the next stage.

The promotion rule can be written as

$$\lambda \text{ is promoted from stage } s \iff c_s(\lambda) \leq Q_q(\{c_s(\lambda') : \lambda' \in \mathcal{C}_s\}),$$

where $c_s(\lambda)$ is the cost of configuration λ at stage s , $\mathcal{C}_s$ is the set of configurations evaluated at stage s , and Q_q is the cost threshold for the configured promotion quantile.

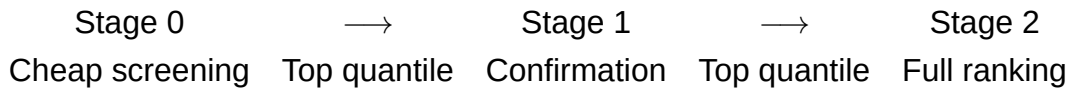


Figure 4.2: Simplified promotion flow in the three-stage fidelity schedule

The schedule spends the largest computational cost on configurations that were already competitive under cheaper approximations. This avoids applying full-fidelity evaluation to every sampled configuration and makes the search more efficient.

4.6.5 Surrogate-Guided Candidate Selection

The search space used by the AutoML engine contains categorical, continuous, integer, and conditional hyperparameters. Since the surrogate model requires numerical input, every sampled ConfigSpace configuration is first converted into a fixed-length numerical vector. Active hyperparameters are encoded using their numerical or encoded categorical values. Missing or inactive dimensions are replaced by finite sentinel values so that all configurations have the same vector length.

Let x_λ denote the vector representation of configuration λ . Since the same configuration can be evaluated at different fidelity levels, the vector is extended with the fidelity information. The augmented input vector is defined as

$$x' = [x_\lambda, s, q, k, b],$$

where s is the fidelity stage, q is the sample fraction, k is the number of cross-validation folds, and b is the model budget. Including these values allows the surrogate model to distinguish between cheap low-fidelity evaluations and expensive full-fidelity evaluations of the same configuration.

During the initial exploration phase, configurations are sampled without relying on the surrogate model. This provides the optimizer with a set of observed configuration–cost pairs. After enough successful evaluations have been collected, the optimizer fits a random-forest regressor using only finite observed costs. Failed evaluations or non-finite results are excluded from the surrogate training data because they do not provide a reliable numerical target.

The random forest acts as an approximation of the unknown cost function. If tree t predicts the cost $\hat{c}_t(x)$ for candidate vector x , then the surrogate mean prediction is estimated as

$$\mu(x) = \frac{1}{T} \sum_{t=1}^T \hat{c}_t(x),$$

where T is the number of trees in the forest. The uncertainty estimate is computed from the disagreement between the trees:

$$\sigma(x) = \sqrt{\frac{1}{T} \sum_{t=1}^T (\hat{c}_t(x) - \mu(x))^2}.$$

A small value of $\sigma(x)$ means that the trees give similar predictions, while a larger value means that the trees disagree more strongly. Therefore, $\mu(x)$ estimates the expected cost of a candidate, and $\sigma(x)$ estimates the uncertainty of that prediction.

The optimizer then uses expected improvement to rank candidate configurations. Because the objective is to minimize cost, improvement is measured relative to the best observed cost c^+ . For a candidate x , the predicted improvement is

$$I(x) = c^+ - \mu(x) - \xi,$$

where ξ is a small exploration parameter. A larger value of ξ encourages the optimizer to prefer candidates that are more likely to produce a clear improvement rather than only a small predicted gain.

The standardized improvement is

$$z(x) = \frac{I(x)}{\sigma(x)}.$$

The expected improvement is then computed as

$$\text{EI}(x) = I(x)\Phi(z) + \sigma(x)\phi(z),$$

where Φ is the standard normal cumulative distribution function and ϕ is the standard normal probability density function. The first term rewards candidates with a low predicted cost, while the second term rewards candidates with higher predictive uncertainty. Therefore, expected improvement balances exploitation and exploration.

Table 4.5: Main quantities used in surrogate-guided candidate selection

Symbol	Meaning
x_λ	Numerical vector representation of configuration λ .
x'	Augmented vector containing configuration and fidelity information.
$\mu(x)$	Mean cost predicted by the random-forest surrogate.
$\sigma(x)$	Prediction uncertainty estimated from disagreement between trees.
c^+	Best observed cost so far.
ξ	Exploration parameter used in expected improvement.
$\text{EI}(x)$	Expected improvement score used to rank candidate configurations.

In each surrogate-guided step, the optimizer samples a pool of candidate configurations from the conditional search space. The candidates are converted to numerical vectors, duplicate vectors are removed, and the remaining unseen candidates are scored using expected improvement. The candidate with the highest expected improvement is scheduled for evaluation.

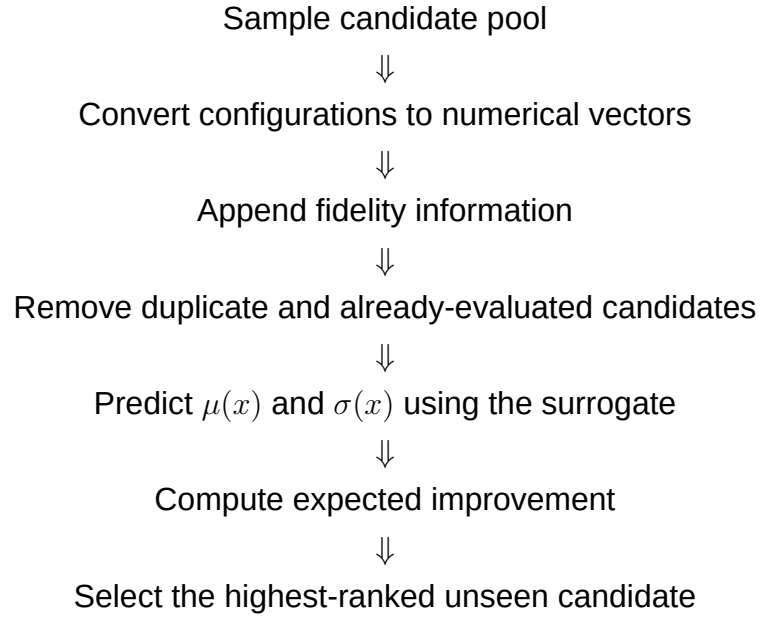


Figure 4.3: Surrogate-guided candidate-selection workflow

This procedure avoids exhaustive enumeration of the mixed conditional search space. Instead of evaluating every possible configuration, the optimizer uses the surrogate model to focus the search on configurations that are either predicted to perform well or have enough uncertainty to be worth exploring.

4.6.6 Warm-Start Selection

Warm-start selection gives the optimizer an informed starting point by reusing configurations that performed well on related historical datasets. For a new dataset D , the engine first computes a task-specific metafeature vector m_D . Historical datasets are represented by stored metafeature vectors m_i , together with their previously successful configurations.

The similarity between the new dataset and each historical dataset is measured using a normalized distance. Let r be a scaling vector for the metafeature dimensions. The distance between the current dataset D and a historical dataset i is

$$d(D, i) = \left\| \frac{m_D - m_i}{r} \right\|_2.$$

The normalization prevents large-scale metafeatures, such as the number of samples or features, from dominating smaller-scale metafeatures. Historical datasets are then ranked by increasing distance, so datasets with smaller distances are considered more similar to the current task.

After ranking the historical datasets, the engine gathers candidate configurations from the nearest datasets. These configurations are not inserted blindly into the search. First, configurations whose model names are not present in the active model set are removed. This ensures that warm starts respect the current user constraints and dataset compatibility checks. Next, configurations are deduplicated using their model name and parameter signature, so repeated historical suggestions do not occupy multiple positions in the initial queue.

Each remaining historical configuration is then converted into a valid ConfigSpace configuration for the current search space. During this conversion, invalid or unsupported parameters are skipped safely. This can happen, for example, when the current version of a component does not contain a parameter that existed in a stored historical configuration, or when the current active space has stricter bounds. A failed conversion does not stop the search; it only removes that warm-start candidate.

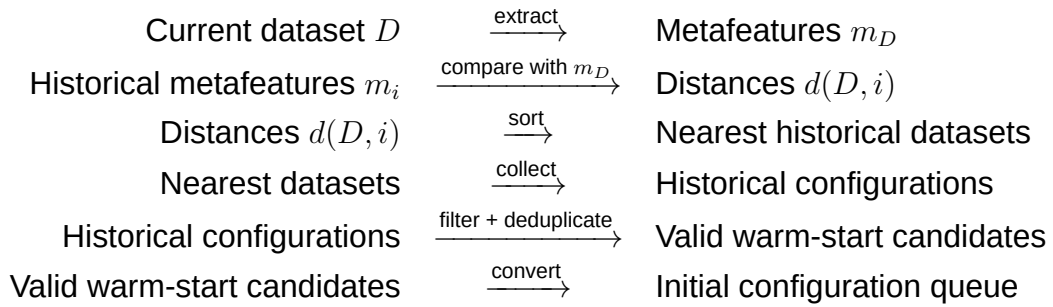


Figure 4.4: Warm-start transfer from similar historical datasets

The number of historical datasets used, the number of configurations taken from each dataset, and the total number of warm-start configurations are bounded by user parameters. These bounds keep the warm-start stage controlled and prevent historical suggestions from dominating the search.

Warm starts are inserted into the initial configuration queue before uninformed random samples. However, they are still evaluated on the current dataset using the same validation and cost-conversion procedure as all other configurations. Therefore, warm starts only guide the early search; they do not bypass evaluation and cannot directly force the final selected model.

4.6.7 Parallel Execution and Timeout Control

The optimizer may suggest a batch of trials in one scheduling step. A batch can contain promoted configurations from earlier fidelity stages, warm-start configurations from historical datasets, and newly proposed configurations selected either by the surrogate model or by random sampling. This batching allows the search to use multiple CPU

cores while keeping the optimizer logic centralized in the parent process.

For parallel execution, each trial is executed in a separate worker process. The worker receives the configuration, builds the corresponding estimator, evaluates it, and returns an `EvaluationResult` through a process-safe queue. The parent process does not wait passively for all workers to finish. Instead, it periodically checks the worker queues, collects completed results, and monitors the elapsed time of each running process.

If a worker process exceeds the configured `per_run_time_limit`, the parent process terminates it and records the trial with a timeout status. The timeout result is stored in the run history in the same way as other failed evaluations, together with a failure cost. This ensures that one slow or non-terminating estimator does not block the complete AutoML search.

The timeout mechanism can be summarized as follows:

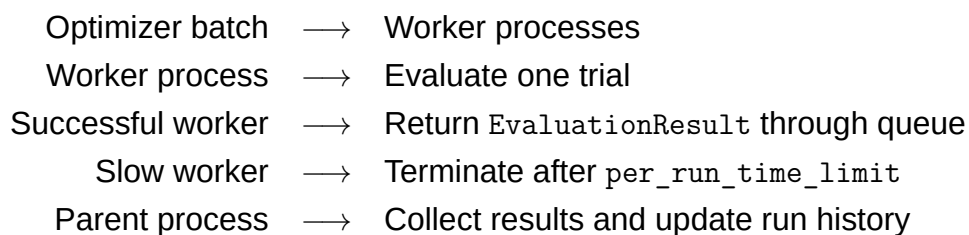


Figure 4.5: Process-level parallel execution and timeout handling

Process-level execution is used instead of thread-level execution because some estimators cannot always be interrupted reliably once they enter a long-running fit procedure. Terminating a separate process is more robust than attempting to stop a thread inside the same Python process. This is especially useful when an estimator hangs, becomes extremely slow, or calls compiled code that does not regularly return control to Python.

Overall, this design improves robustness and resource control. The optimizer can continue receiving valid results even when some trials fail or time out, and the caller can control the amount of parallel work performed by the search.

4.6.8 Final Model and Ensemble Selection

After the search phase finishes, the engine selects the strongest configuration evaluated at full fidelity. This configuration is rebuilt from its stored model name, preprocessing choice, and hyperparameters, and then fitted on all available training data. This final refitting step is necessary because the search phase estimates performance using validation folds, while the final model should use the complete

training set before being returned to the user.

If ensemble construction is disabled, the refitted best individual model is returned directly. If ensembles are enabled, the engine considers successful candidates from the run history as possible ensemble members. Failed trials, timed-out trials, and configurations without valid fitted estimators are ignored. This ensures that the ensemble is built only from models that were evaluated successfully during the search.

The ensemble construction stage supports two strategies: voting and stacking. For voting, the engine selects a bounded set of competitive estimators from the run history. These estimators are combined using a task-specific voting model. For classification, the voting ensemble combines class predictions or class probabilities. For regression, it combines numeric predictions. Voting is simple and robust because it does not train an additional meta-model.

Stacking uses a more expressive structure. Instead of directly averaging the base-model outputs, it trains a meta-model that learns how to combine them. The engine first considers candidate diversity before selecting base estimators, so the stack is not formed only from many near-identical models. Each selected base estimator is wrapped in bagging to improve stability. The base estimators then produce out-of-fold predictions, which are used as training features for the meta-model.

Forward selection is used to choose the final set of base estimators for the stacking ensemble. The procedure starts with the strongest single candidate. Additional candidates are tested one at a time. A candidate is kept if adding it improves the validation score of the meta-model. The process stops when no remaining candidate improves the score, or when the configured ensemble size is reached. However, the minimum base-model requirement is also respected, so the engine can include enough base estimators to form a valid stack even if early improvements are small.

For classification, each base estimator may output a probability vector over the class labels. Before these outputs are passed to the meta-model, the probability columns are aligned according to the global class order. This avoids a common problem where different classifiers store class labels in different orders. If the columns were not aligned, the meta-model could interpret the probability of one class as if it belonged to another class. After alignment, the probability blocks from all base estimators are concatenated and passed to logistic regression as the classification meta-model.

For regression, the stacking representation is simpler. Each base estimator contributes one numeric prediction column. These columns are concatenated to form the meta-feature matrix, and ridge regression is used as the regression meta-model.

Ridge regression is suitable here because it can combine correlated base predictions while controlling the size of the learned coefficients.

The best individual model is always available as a safe fallback. The ensemble is only returned when it performs at least as well as this individual model under the same validation metric. The final decision can be written as

$$\text{Final model} = \begin{cases} \text{ensemble,} & \text{if } S_{\text{ens}} \geq S_{\text{best}}, \\ \text{best individual model,} & \text{otherwise,} \end{cases}$$

where S_{ens} is the validation score of the ensemble and S_{best} is the validation score of the strongest individual model. This rule prevents the system from returning a more complex ensemble unless it provides at least the same predictive quality as the best single model. As a result, ensemble construction can improve performance when the base models complement each other, while still preserving the simpler individual model when the ensemble does not provide a measurable benefit.

4.7 Chapter Summary

This chapter presented Accelera AutoML. It described the module overview and assumptions, showed the system architecture and block diagram, listed the functional and non-functional requirements, and explained the implemented AutoML approach. The module design combines conditional configuration-space construction, cross-validated evaluation, staged fidelity, surrogate-guided candidate selection, meta-learning warm starts, process-level timeout control, and final ensemble selection.

Chapter 5: System Testing and Verification

This chapter explains how the correct operation and performance of Accelera AutoML were tested and verified. It describes the benchmark setup, testing strategy, comparative classification and regression results, reproduction assets, and threats to validity. The goal is to show both that the implementation works as intended and that its measured behavior can be inspected and reproduced.

5.1 Testing Setup

The experiments compare Accelera AutoML with Auto-sklearn, FLAML, and TPOT on supervised tabular classification and regression tasks. The evaluation asks two practical questions: whether the systems attain comparable predictive quality, and how much wall-clock time they require to do so. Classification is evaluated primarily by accuracy, while regression is evaluated by the coefficient of determination (R^2), root mean squared error (RMSE), and mean absolute error (MAE). Runtime is reported in seconds for both task types.

5.2 Testing Plan and Strategy

The classification experiments were executed through the open-source AutoMLBenchmark (AMLB) runner, available at <https://github.com/openml/automlbenchmark.git>. They use the `flaml_paper_39` classification suite and the `flaml_paper_1h` constraint. The reported classification evaluation covers 12 datasets. The regression benchmark was built from the Penn Machine Learning Benchmarks (PMLB) repository, available at <https://github.com/EpistasisLab/pmlb.git>, and was also executed through AMLB. The reported regression evaluation covers 10 datasets and uses the `pmlb_regression` suite with the `pmlb_30m` constraint.

All systems were run through the same benchmark interface. The recorded setup used one fold and one allocated core per task. The regression time budget was 1,800 seconds per dataset; the classification constraint allowed up to one hour.

5.3 Comparative Results to Previous Work: Classification

Table 5.1 summarizes the classification results stored with the project. Auto-sklearn obtained the highest mean accuracy at 0.8842. Accelera reached a mean accuracy of 0.8791, a difference of about 0.0051, while recording the lowest mean duration at 1,201.7 seconds. Its mean runtime was approximately one third of the runtimes recorded for the other three systems. Accelera also had the largest number of recorded speed wins.

Table 5.1: Classification benchmark summary.

System	Mean accuracy	Mean time (s)	Accuracy wins	Speed wins
Auto-sklearn	0.884178	3615.71	3	3
FLAML	0.881427	3655.63	6	0
Accelera	0.879076	1201.72	6	15
TPOT	0.877654	3953.78	8	1

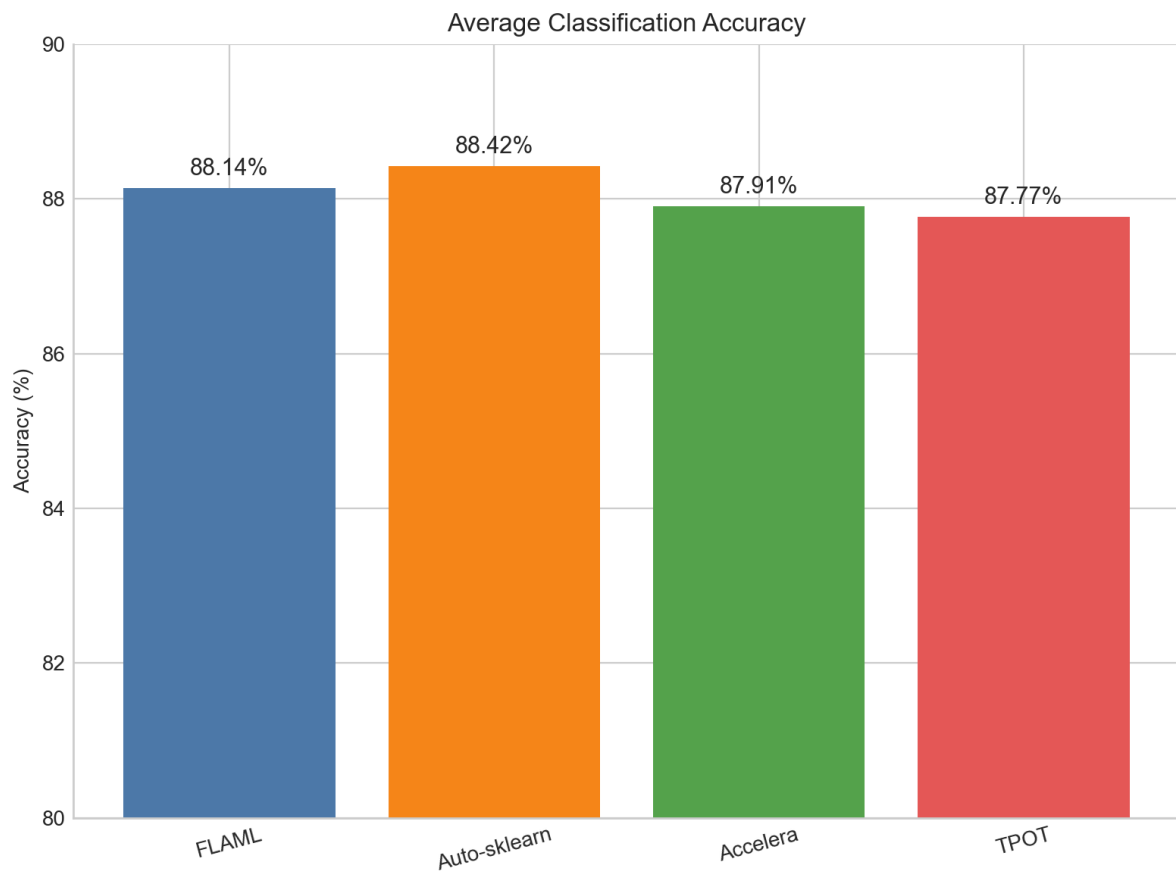


Figure 5.1: Mean classification accuracy for the evaluated systems.

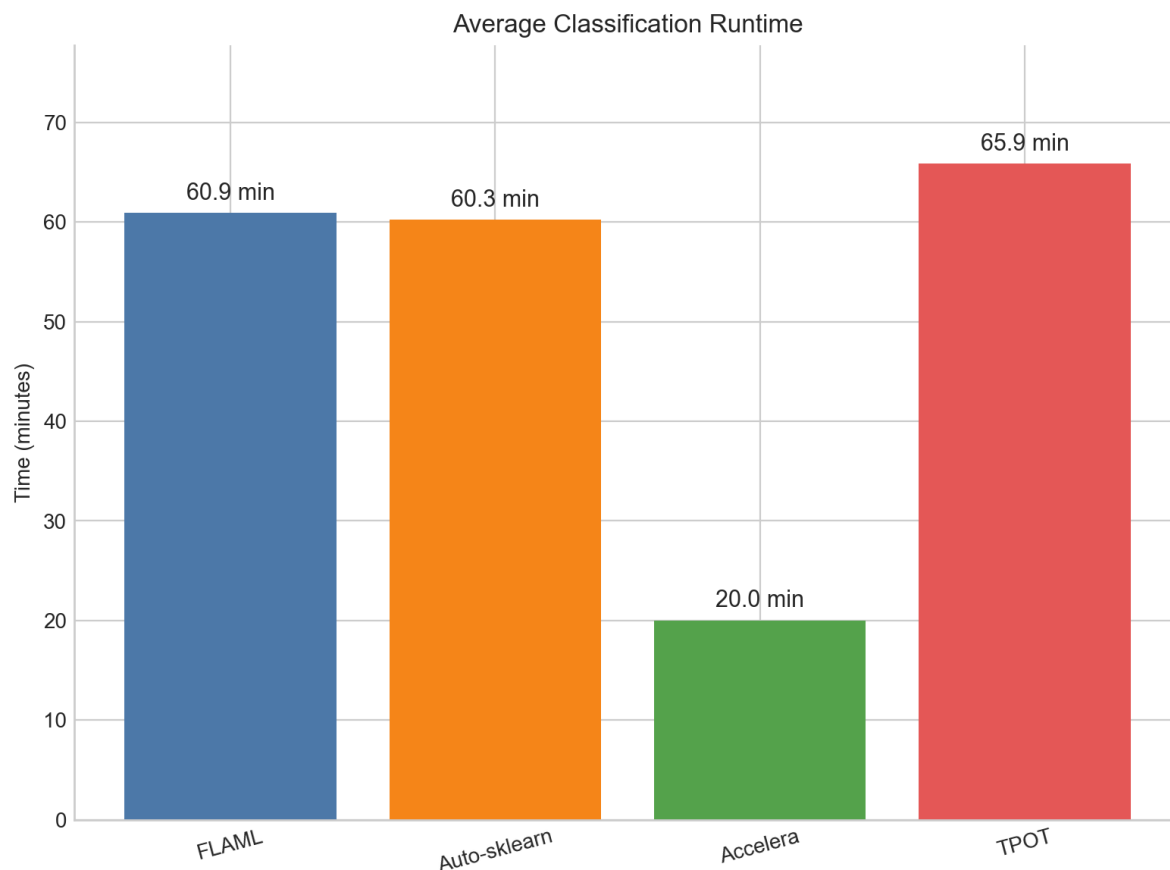
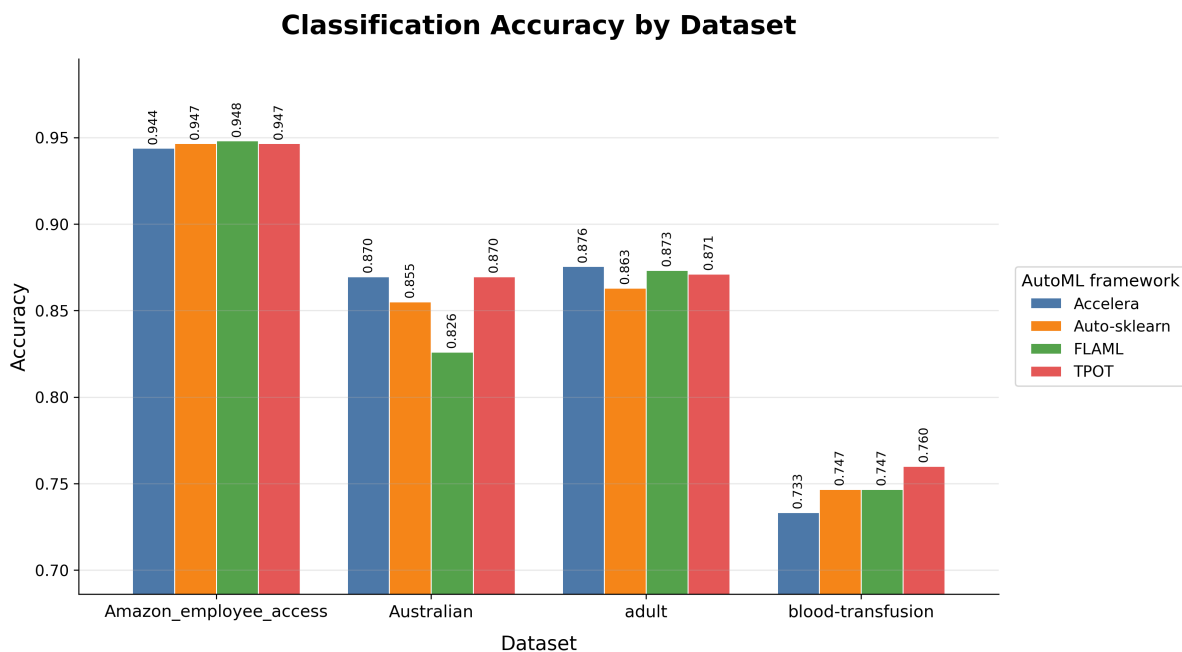
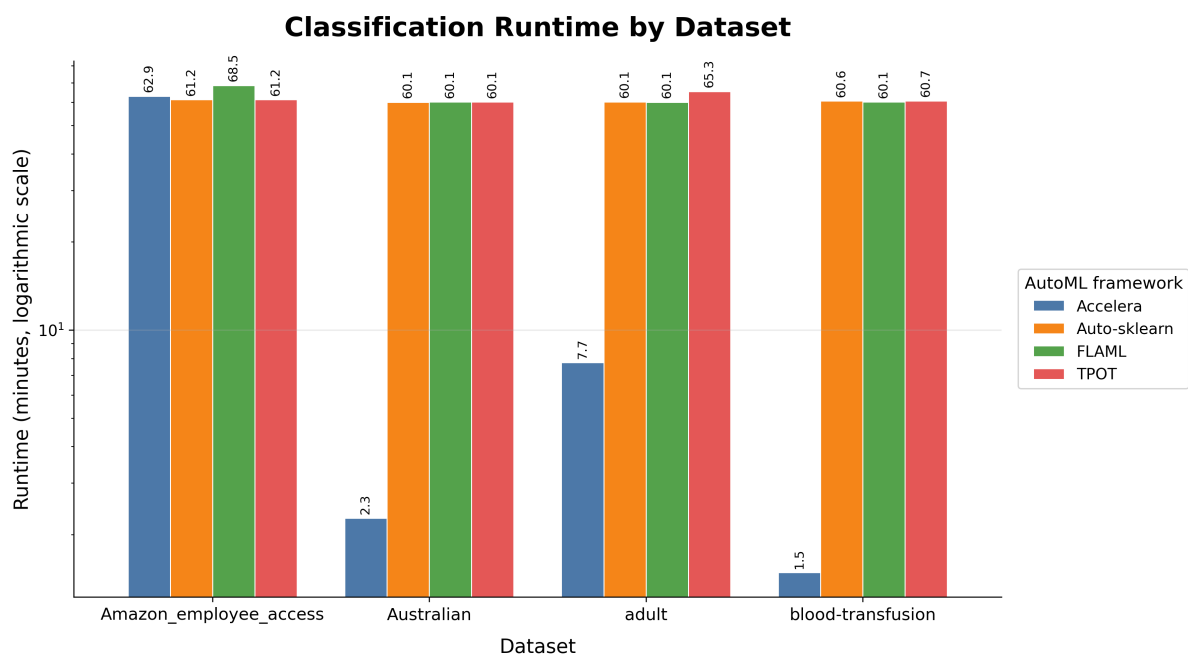


Figure 5.2: Mean classification runtime for the evaluated systems.



The four datasets shown are sampled from the full set of tested datasets.

Figure 5.3: Per-dataset classification accuracy on common datasets.



The four datasets shown are sampled from the full set of tested datasets.

Figure 5.4: Per-dataset classification duration on common datasets.

5.4 Comparative Results to Previous Work: Regression

Table 5.2 reports the updated regression summary over ten datasets. Auto-sklearn obtained the highest mean R^2 at 0.5609, followed closely by TPOT at 0.5592. Accelera reached 0.5294 and recorded the lowest mean runtime at 892.64 seconds, compared with 1,820–1,864 seconds for the competing systems. Accelera also achieved six speed wins, the most of any evaluated system.

Table 5.2: Updated regression benchmark summary over ten datasets.

System	Mean R^2	Median R^2	Mean time (s)	R^2 wins	RMSE wins	MAE wins	Speed wins
Auto-sklearn	0.560947	0.494362	1820.18	2	2	2	2
TPOT	0.559238	0.493144	1849.70	2	2	2	0
Accelera	0.529389	0.432577	892.64	2	2	2	6
FLAML	0.522743	0.497530	1863.85	4	4	4	2

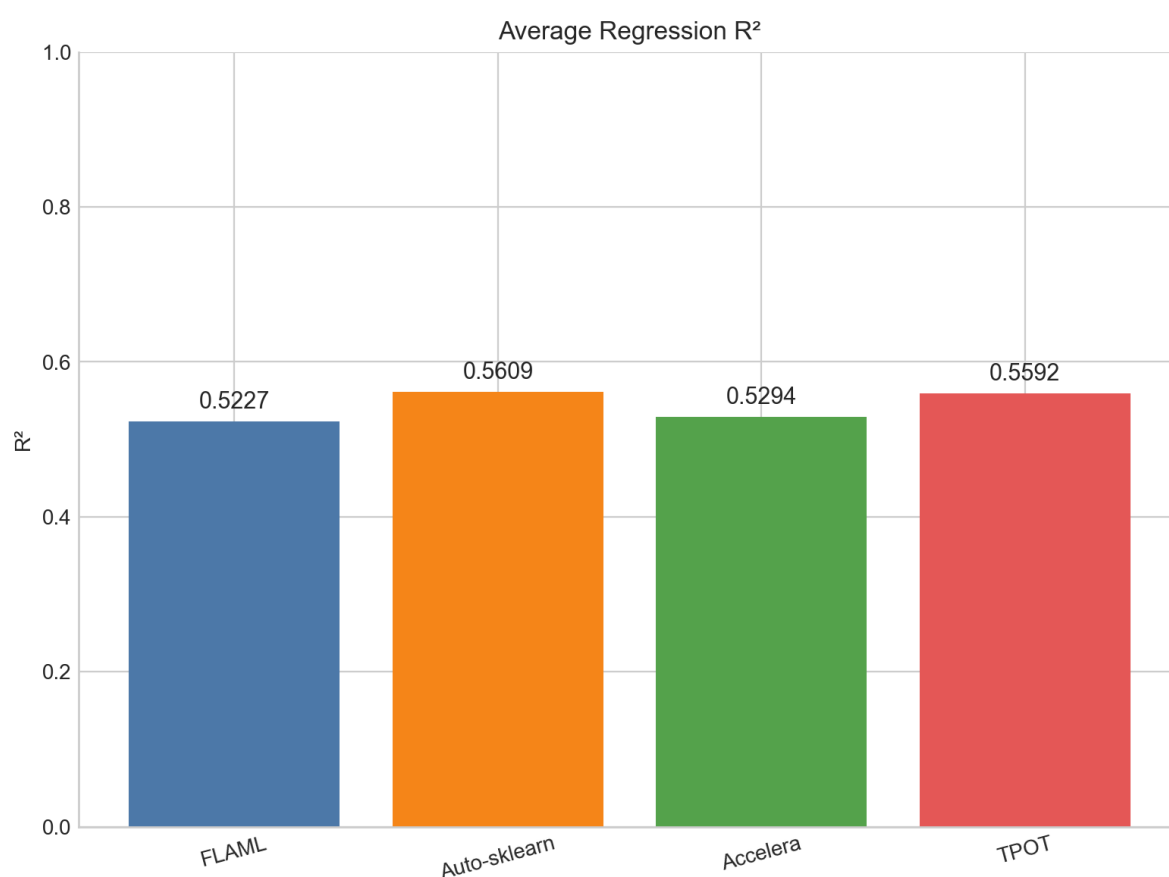


Figure 5.5: Mean regression R^2 for the evaluated systems.

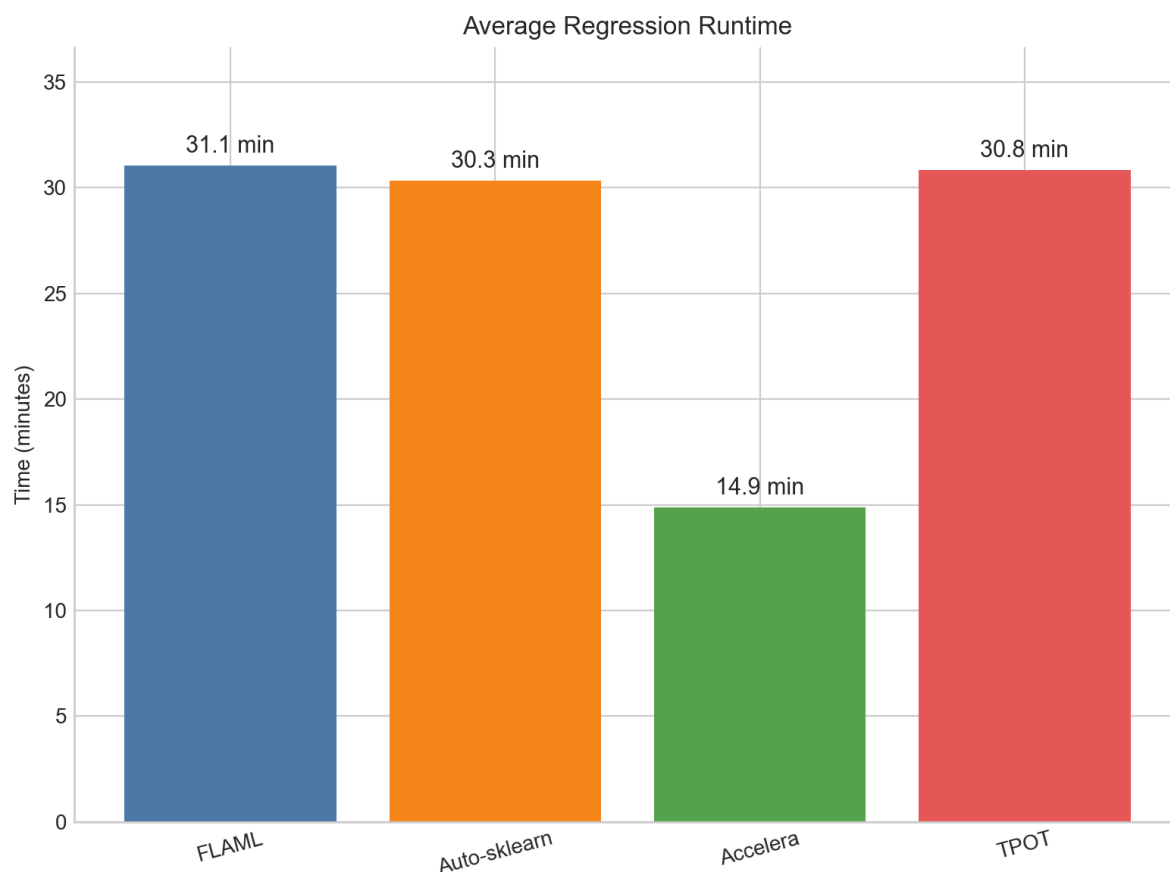
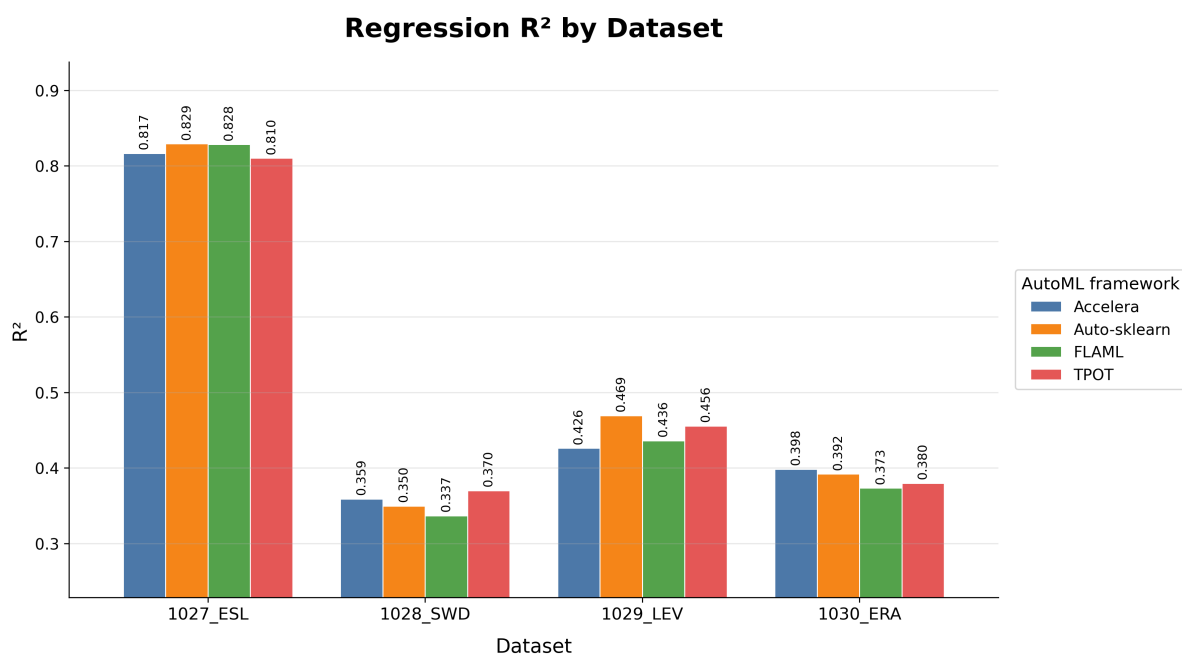
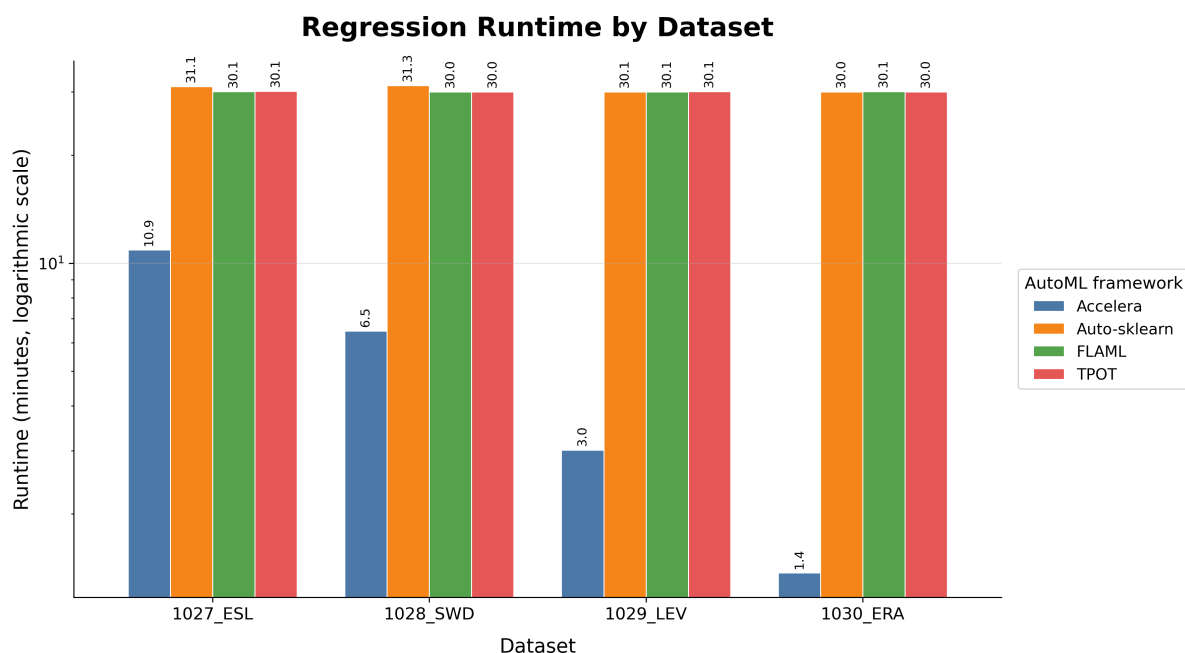


Figure 5.6: Mean regression runtime for the evaluated systems.



The four datasets shown are sampled from the full set of tested datasets.

Figure 5.7: Per-dataset regression R^2 on common datasets.



The four datasets shown are sampled from the full set of tested datasets.

Figure 5.8: Per-dataset regression duration on common datasets.

5.5 Result Reproduction

The benchmark experiments can be reproduced using the instructions provided in the project repository. These instructions describe the required environment setup and the commands used to rerun the classification and regression benchmarks. The generated figures, CSV summaries, and raw benchmark outputs are stored with the project results, allowing the reported values to be checked against future reruns. The reproduction guide is available at [setup/reproduce_results.md](#).

5.6 Chapter Summary

Across the stored experiments, Accelera achieved predictive results close to the strongest competing framework while requiring markedly less mean runtime. The classification results show a small accuracy difference relative to Auto-sklearn, and the regression results show a modest R^2 difference relative to Auto-sklearn, accompanied in both cases by substantial runtime reductions. The complete stored artifacts and reproduction instructions are retained with the implementation so that the reported measurements can be inspected and rerun.

Chapter 6: Conclusions and Future Work

This chapter summarizes the project outcomes and reflects on the experience gained during development. It reviews the main challenges faced during the implementation and evaluation of Accelera AutoML, states the conclusions reached from the reported results, and identifies future extensions that can improve the module.

6.1 Faced Challenges

The main challenges were balancing predictive quality with runtime, preserving fair evaluation across different estimator families, isolating trial failures, and keeping the search process inspectable. Multi-fidelity evaluation reduced some search cost, but it required careful promotion rules so that cheap early signals did not dominate final model selection unfairly.

6.2 Gained Experience

The project provided experience in AutoML system design, conditional search spaces, Bayesian optimization, cross-validation protocols, process-level timeouts, Python package resource handling, and ensemble construction. It also showed the importance of benchmark reproducibility and of exposing run history instead of returning only a final fitted model.

6.3 Conclusions

Accelera AutoML provides an Accelera-integrated AutoML module for supervised tabular classification and regression. It combines conditional model spaces, staged fidelity, surrogate-guided search, meta-learning warm starts, timeout isolation, and voting or stacked ensembles behind a scikit-learn-style public API. The stored experiments show predictive performance close to established AutoML systems while reducing mean runtime in the recorded benchmark runs.

6.4 Future Work

Future work should include repeated benchmark folds, broader datasets, stronger reporting of failed tasks, expanded support for categorical preprocessing, additional model families, time-series validation modes, distributed execution, and a completed public benchmark reproduction repository.

Appendix A: Development Platforms and Tools

A.1 Development Platform

The implementation uses Python, scikit-learn, ConfigSpace-style conditional configuration handling, gradient-boosting libraries, pytest-based verification, and the AutoMLBenchmark runner for stored benchmark results.

A.2 Software Tools

A description of each main software tool or package used by the implementation is provided in a separate subsection.

A.2.1 ConfigSpace

The `ConfigSpace` package is used for ConfigSpace-style conditional configuration handling. In Accelerate AutoML, it represents the AutoML search space as a set of typed hyperparameters with activation conditions. A global `model_name` hyperparameter selects the active estimator family, while model-specific preprocessing and hyperparameter choices are enabled only when their parent model or option is active. This prevents invalid combinations from being sampled and allows the optimizer, warm-start loader, and evaluator to work with a single valid configuration object.

Appendix B: Use Cases

The main use cases of Accelera AutoML are:

- fitting a classifier and obtaining predicted labels or class probabilities;
- fitting a regressor and obtaining numeric predictions;
- restricting the search to an allowed subset of model families;
- selecting the evaluation metric and the number of cross-validation folds;
- limiting the total search budget or the timeout for each individual trial;
- enabling or disabling meta-learning warm starts and ensemble construction;
- inspecting the best score, leaderboard, and complete run history.

Appendix C: User Guide

Accelera AutoML follows a scikit-learn-style workflow. The user first imports either `AutoMLClassifier` or `AutoMLRegressor`, creates an estimator with the required search settings, fits it on the training data, and then uses the fitted model for prediction, scoring, or leaderboard inspection.

The following example shows the basic classification workflow:

```
from accelera_automl import AutoMLClassifier

model = AutoMLClassifier(
    time_budget=1800,
    cv=5,
    scoring="accuracy",
    use_ensemble=True,
    random_state=42,
)

model.fit(X_train, y_train)

predictions = model.predict(X_test)
score = model.score(X_test, y_test)
leaderboard = model.return_leaderboard()
```

In this example, the search is limited to 1,800 seconds, five-fold cross-validation is used, and accuracy is selected as the optimization metric. After fitting, the model can generate predictions, evaluate performance on test data, and return the stored leaderboard for inspection.

Appendix D: Code Documentation

The main source package is located under `accelera/src/accelera_automl/`. The public API wrappers are under `accelera/api/accelera_automl/`. This appendix maps the design described in Chapter 5 to the source-code organization.

Table D.1: Main implementation units of Accelera AutoML

File or directory	Responsibility
<code>base.py</code>	Shared estimator lifecycle, validation, fitted state, leaderboard, and run-history access.
<code>classification.py</code>	Classification defaults and task-specific engine construction.
<code>regression.py</code>	Regression defaults and task-specific engine construction.
<code>core/automl.py</code>	AutoML orchestration, final model fitting, compatibility checks, and ensemble selection.
<code>components/</code>	Model registries, hyperparameter definitions, activation conditions, and estimator factories.
<code>configspace_search_space.py</code>	Task-specific conditional search-space assembly and configuration conversion.
<code>evaluation.py</code>	Fidelity-aware model construction, preprocessing comparison, cross-validation, and metric-to-cost conversion.
<code>optimization/smac_optimizer.py</code>	Trial scheduling, surrogate modelling, expected improvement, promotions, parallel workers, and timeout control.
<code>meta_learning/</code>	Metafeature extraction, similarity ranking, and warm-start retrieval.
<code>meta_learning_data/</code>	Packaged historical classification and regression metadata used for warm starts.
<code>stacked_ensemble.py</code>	Classification adapters, bagging, forward selection, and stacked classification.
<code>stacked_ensemble_regression.py</code>	Regression stacking and forward selection.

The public estimator lifecycle starts when the user calls `fit`. The estimator validates X and y , resets previous fitted state, builds the task-specific engine, runs the search, and stores the returned result. Prediction methods validate the input and delegate to the selected final model. Calling prediction before fitting raises an explicit runtime error.

The stored result includes the best individual model, the optional ensemble, the selected final model, the best score, the leaderboard, and the detailed run-history records. These records make the module inspectable because they preserve both successful and failed trials.

Appendix E: Feasibility Study

The proposed module is technically feasible because it is built on established Python machine-learning libraries such as scikit-learn and can be integrated within the existing Accelera package structure. The implementation does not require specialized hardware and can run on standard CPU-based environments, although larger search budgets and larger datasets require more computation time.

From an operational perspective, the module is feasible because it provides a simple scikit-learn-style interface. Users can create an AutoML classifier or regressor, call `fit`, and then use standard methods such as `predict`, `score`, and `return_leaderboard`. This makes the system suitable for users who want automated model selection without manually testing many algorithms and hyperparameter settings.

The project is also economically feasible because it mainly depends on open-source software tools and does not require paid cloud services or commercial AutoML platforms. Reusing the same module across multiple classification and regression tasks can reduce the manual effort needed for model selection, tuning, and comparison.

The main limitation is computational cost. AutoML search can be time-consuming, especially when many models, folds, and trials are evaluated. However, this cost is controlled through time budgets, per-trial timeouts, multi-fidelity evaluation, and configurable search settings. Therefore, the project is feasible as a practical AutoML extension for the Accelera framework.

Bibliography

- [1] Chris Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013.
- [2] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*, 2015.
- [3] Randal S. Olson, Nathan Bartley, Ryan J. Urbanowicz, and Jason H. Moore. Evaluation of a tree-based pipeline optimization tool for automating data science. In *Applications of Evolutionary Computation*, 2016.
- [4] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. FLAML: A fast and lightweight AutoML library. In *Proceedings of Machine Learning and Systems*, 2021.
- [5] Nick Erickson et al. AutoGluon-Tabular: Robust and accurate AutoML for structured data. *arXiv preprint arXiv:2003.06505*, 2020.
- [6] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *Learning and Intelligent Optimization*, 2011.
- [7] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [8] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.